

EXPERIMENT 5

InterProcess Communication using Pipes

OBJECTIVE:

- Learn and Understand InterProcess Communication using implementation of Pipes

BACKGROUND:

Pipes:

Ordinary pipes allow two processes to communicate in standard producer consumer fashion: the producer writes to one end of the pipe (the **write-end**) and the consumer reads from the other end (the **read-end**). **As a result, ordinary** pipes are unidirectional, allowing only one- way communication. If two-way communication is required, two pipes must be used, with each pipe sending data in a different direction.

- A pipe has a read end and a write end.
- Data written to the write end of a pipe can be read from the read end of the pipe.

Creating an Ordinary Pipe:

On UNIX and LINUX systems, ordinary pipes are constructed using the function

- `int pipe(int fd[2])` – creates a pipe
- returns two file descriptors, `fd[0]`, `fd[1]`.
- `Fd[0]` is the read-end of the pipe.
- `Fd[1]` is the write-end of the pipe.
- `Fd[0]` is opened for reading
- `Fd[1]` is opened for writing. `Pipe()` sets 0 on success, -1 on failure and sets `errno` accordingly.
- The standard programming model is that after the pipe has been set up, two (or more) cooperative processes will be created using `read()` and `write()`.
- Pipes opened with `pipe()` should be closed with `close(int fd)`.

Example 1:

```
#include <unistd.h>
#include <stdio.h>
#include <string.h>

int main()
{
    int pdes[2];
    char buf[100];

    pipe(pdes); // Create pipe

    if (fork() == 0) // Child
    {
        close(pdes[1]); // Close unused write end
        read(pdes[0], buf, sizeof(buf)); // Read from parent
        printf("Child received: %s\n", buf);
        close(pdes[0]); // Close read end
    }
    else // Parent
    {
        close(pdes[0]); // Close unused read end
        const char *msg = "Hello from parent!";
        write(pdes[1], msg, strlen(msg) + 1); // Write to child
        close(pdes[1]); // Close write end
    }

    return 0;
}
```

When pipe() System Call Fails:

The pipe() system call fails for many reasons, including the following:

- At least two slots are not empty in the FDT – too many files or pipes are open in the processes. (Each process has a File Descriptor Table (FDT) that tracks open files, pipes) sockets, etc.
- Buffer space not available in the kernel.

In-lab Questions:

Q1. Design a program using ordinary pipes in which one process sends a string message to a second process, and the second process reverses the case of each character in the message and sends it back to the first process. For example, if the first process sends the message Hi There, the second process will return hI tHERE. This will require using two pipes, one for sending the original message from the first to the second process, and the other for sending the modified message from the second back to the first process.

